

Assignment

```
import java.io.*;  
import java.util.*;
```

```
class Book implements Comparable <Book> {  
    int bookId;  
    String title;  
    String author;  
    String category;  
    boolean issued;
```

```
    Book(int id, String t, String a, String c) {  
        bookId = id;  
        title = t;  
        author = a;  
        category = c;  
        issued = false;  
    }
```

```
    void markIssued () { issued = true; }
```

```
    void markReturned () { issued = false; }
```

```
    public int compareTo (Book b) {  
        return title.compareToIgnoreCase (b.title);  
    }
```

```
    String toString () {  
        return bookId + "," + title + "," + author +  
            "," + category + "," + issued;
```

```

Static Book from CSV(string s){
    string[] p = s.split(",");
    if (p.length < 5) return null;
    Book b = new Book(Integer.parseInt(p[0]),
    p[1], p[2], p[3]);
    b.issued = Boolean.parseBoolean(p[4]);
    return b;
}
}

```

```

Class Member {
    int memberId;
    String name;
    List<Integer> issuedBooks;
}

```

```

Member(int id, String n) {
    memberId = id;
    name = n;
    issuedBooks = new ArrayList<>();
}

```

```

Void addBook(int id) { issuedBooks.add(id); }
Void returnBook(int id) {
    issuedBooks.remove(Integer.valueOf(id)); }

```

```

String toCSV() {
    String Builder sb = new String Builder();
    sb.append(memberId).append(",").append(name).
    append(",");
    for (int i = 0; i < issuedBooks.size(); i++) {

```

```

        if (i > 0) cb.append(",");
        sb.append(issuedBooks.get(i));
    }
    return sb.toString();
}

```

```

static Member from CSV(String s) {
    String p = s.split(",");
    Member m = new
    Member(Integer.parseInt(p[0]), p[1]);
    if (p.length == 3 && !p[2].isEmpty()) {
        for (String x : p[2].split(",")) {
            m.issuedBooks.add(Integer.parseInt(x));
        }
    }
    return m;
}
}

```

```

class Library Management System {
    Map<Integer, Book> books = new
    HashMap<>();
    Map<Integer, Member> member = new
    HashMap<>();
    Scanner sc = new Scanner(System.in);

```

```

    String bookFile = "books.txt";
    String membersFile = "members.txt";

```

```
Public static void main(String[] args) {
    new Library Management System().run();
}
```

```
void run() {
    loadFromFile();
    mainMenu();
    saveToFile();
}
```

```
void mainMenu() {
    while (true) {
        System.out.println("\n--- City Library  
Digital Management System ---");
        System.out.println("1. Add Book");
        System.out.println("2. Add Member");
        System.out.println("3. Issue Book");
        System.out.println("4. Return Book");
        System.out.println("5. Search Book");
        System.out.println("6. Sort Book");
        System.out.println("7. Exit");
```

```
int choice = readInt("Enter choice: ");
```

```
switch (choice) {
    case 1: addBook(); break;
    case 2: addMember(); break;
    case 3: issueBook(); break;
    case 4: returnBook(); break;
    case 5: searchBooks(); break;
    case 6: sortBooks(); break;
```

```

        case 7: saveToFile();
        System.out.println("Goodbye!"); return;
        default: System.out.println("Invalid
        choice.");
    }
}
}

```

```

int nextBookId() {
    return book.keySet().stream().mapToInt(i
    -> i).max().orElse(100)+1;
}

```

```

int nextMemberId() {
    return
    member.keySet().stream().mapToInt(i->
    i).max().orElse(200)+1;
}

```

```

void addBook() {
    String t = readLine("Enter Title:");
    String a = readLine("Enter Author:");
    String c = readLine("Enter Category:");

```

```

    int id = nextBookId();
    books.put(id, new Book(id, t, a, c));

```

```

    System.out.println("Book added. ID: "+
    id);
}

```

```

void addMember() {
    String n = readLine("Enter Member Name:");
    int id = nextMemberId();
    members.put(id, new Member(id, n));
    System.out.println("Member added. ID: " + id);
}

```

```

void issueBook() {
    int bid = readInt("Enter Book ID:");
    Book b = books.get(bid);

    if (b == null) {
        System.out.println("Book not found.");
        return;
    }
    if (b.isIssued()) {
        System.out.println("Book is already issued.");
        return;
    }

    int mid = readInt("Enter Member ID:");
    Member m = members.get(mid);

    if (m == null) {
        System.out.println("Member not found.");
        return;
    }

    b.markIssued();
    m.addBook(bid);

    System.out.println("Book issued.");
}

```

```

void returnBook() {

```

```
int bid = readInt("Enter Book ID: ");
```

```
Book b = books.get(bid);
```

```
if (b == null) { System.out.println("Book not  
found."); return; }
```

```
int mid = readInt("Enter Member ID: ");
```

```
Member m = members.get(mid);
```

```
if (m == null) { System.out.println("Member  
not found."); return; }
```

```
if (!m.issuedBooks.contains(bid)) {  
    System.out.println("This member did  
not issue this book.");  
    return;  
}
```

```
b.markReturned();
```

```
m.returnBook(bid);
```

```
System.out.print("Book returned.");  
}
```

```
void searchBooks() {
```

```
    String q = readLine("Enter search  
keyword (title/author/category):  
").toLowerCase();
```

```
    boolean found = false
```

```
    for (Book b : books.values()) {
```

```

        if (b.title.toLowerCase().contains(q))
            ||
        b.author.toLowerCase().contains(q)
            ||
        b.category.toLowerCase().contains(q)) {
            print Book(b);
            found = true;
        }
    }
}

```

```

    If (!found)
        System.out.println("No books found.");
}

```

```

Void sort Books() {
    List<Books> list = new
    Array List<>(books.values());
}

```

```

    System.out.print("Sort by: 1. Title
    2. Author 3. category");
    int c = readInt("Enter option:");
}

```

```

    If (c == 1) collection.sort(list);
    else if (c == 2)
        list.sort(Comparator.comparing(x -> x.author,
        String.CASE_INSENSITIVE_ORDER));
    else if (c == 3)
        list.sort(Comparator.comparing(x -> x.category,
        String.CASE_INSENSITIVE_ORDER));
}

```

```

    for (Book b: list) print Book(b);
}

```

```
void print Book(Book b) {  
    System.out.println("ID:" + bookId  
        + "| Title:" + .title  
        + "| Author:" + b.author  
        + "| Category:" + b.category  
        + "| Issued:" + b.issued;  
}
```

```
String readLine(String p){  
    System.out.println(p);  
    String s = sc.nextLine();  
    return s.trim();  
}
```

```
int readInt(String p){  
    while(true){  
        try {  
            System.out.println(p);  
            return  
Integer.parseInt(sc.nextLine().trim());  
        } catch (Exception e) {  
            System.out.println("Enter a valid  
Integer.");  
        }  
    }  
}
```

```
void loadFromFile(){  
    try {  
        File f = new File(bookFile);  
        if(!f.exists()) f.createNewFile();  
    }
```

```

        Buffered Reader br = new
        Buffered Reader (new File Reader (f));
        String line;
        while ((line = br.readLine()) != null) {
            Book b = Book.from CSV(line);
            if (b != null) books.put (b.bookId, b);
        }
        br.close ();
    } catch (Exception e) {
        System.out.println ("Error loading
        books.");
    }
}

```

```

try {
    File f = new File (member File);
    if (!f.exists ()) f.createNewFile ();
    Buffered Reader br = new
    Buffered Reader (new File Reader (f));
    String line;
    while ((line = br.readLine()) != null) {
        Member m = Member.from CSV(line);
        if (m != null)
            member.put (m.memberId, m);
    }
    br.close ();
} catch (Exception e) {
    System.out.println ("Error loading
    members.");
}
}

```

```
void saveToFile() {  
    try {  
        BufferedWriter bw = new  
        BufferedWriter(new FileWriter(booksFile));  
        for (Book b: books.values())  
            bw.write(b.toCSV() + "\n");  
        bw.close();  
    } catch (Exception e) {  
        System.out.println("Error saving  
books.");  
    }  
}
```

```
try {  
    BufferedWriter bw = new  
    BufferedWriter(new FileWriter(membersFile));  
    for (Member m: member.values())  
        bw.write(m.toCSV() + "\n");  
    bw.close();  
} catch (Exception e) {  
    System.out.println("Error saving  
members.");  
}  
}
```